DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Servidores web y DNS

Tema 3 — Publicación y ejecución de aplicaciones web



Objetivo de la sesión

Publicar Escaparate y su documentación con Nginx

Qué cambia

Nginx recibe las peticiones HTTP.

El frontend se sirve como ficheros.

Spring Boot conserva la lógica.

Qué comprobaremos

Dos nombres y dos sitios.

Rutas, permisos y cabeceras.

Compresión y caché de recursos.

Nginx recibe las conexiones públicas y entrega cada recurso desde el lugar adecuado.

Contenido estático y lógica de aplicación

Nginx sirve ficheros

HTML, CSS, JavaScript e imágenes.

Localiza el recurso bajo la raíz del sitio y devuelve su contenido.

Spring Boot genera respuestas

Las peticiones a /api/... ejecutan lógica de aplicación.

La aplicación consulta la base de datos cuando lo necesita.

Apache y Nginx pueden servir estáticos y actuar como proxy. En el módulo utilizaremos Nginx también para los siguientes temas.

En esta sesión, /api/ conecta con el backend. El proxy inverso se estudiará después.

Nginx como nueva puerta de entrada



Servicio, configuración y contenido

Servicio web en Compose

```
services:
  web:
    image: nginx:1.30.4-alpine
    ports:
      - "80:80"
    volumes:
      - ../nginx/conf.d:/etc/nginx/conf.d:ro
      - ../nginx/sitio-escaparate/escaparate:/srv/www/escaparate:ro
      - ../nginx/sitio-docs:/srv/www/docs:ro
```

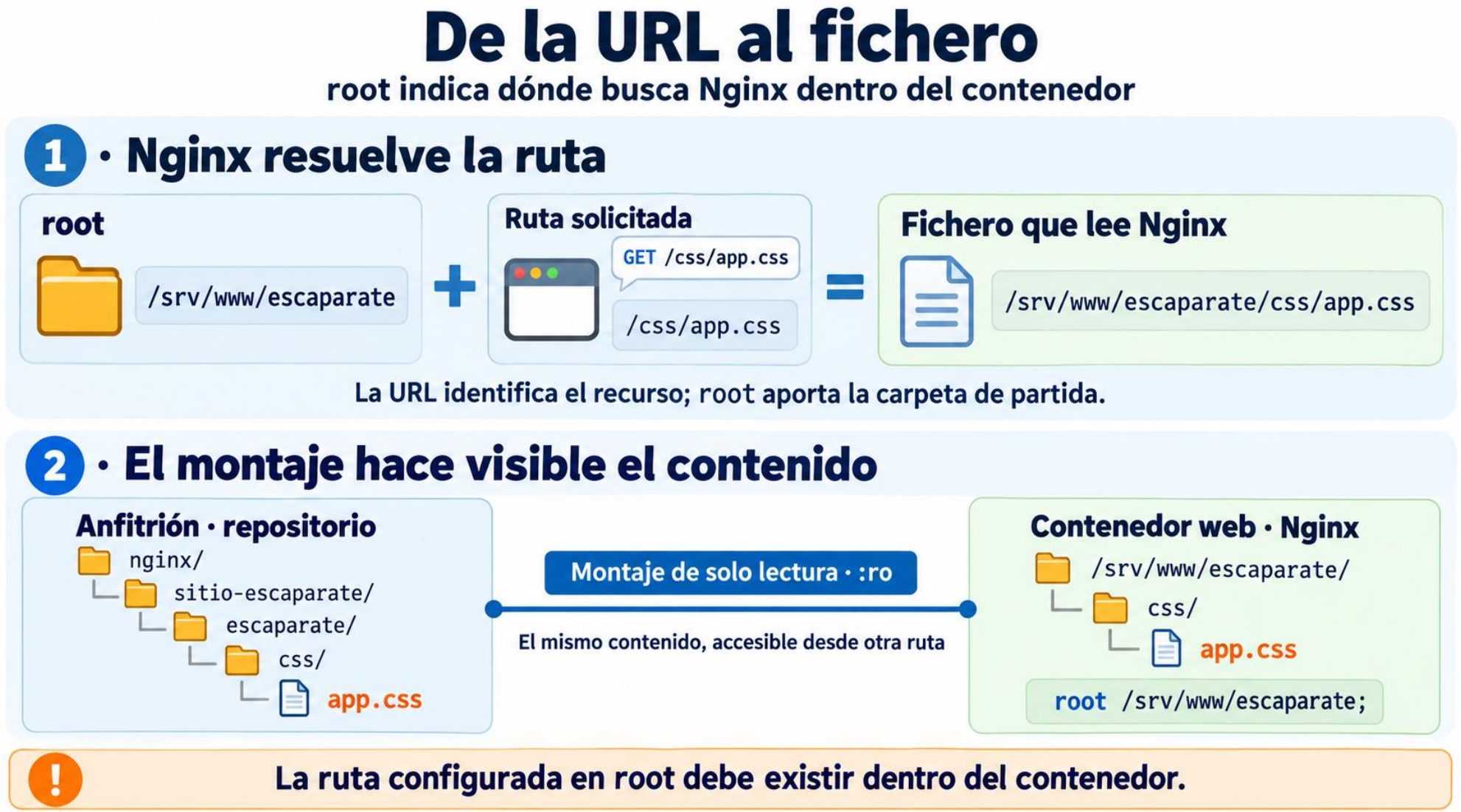
Imagen oficial de Nginx

Configuración versionada

Contenido estático

Los montajes :ro permiten leer la configuración y los ficheros desde el contenedor.

Raíz de documentos y montajes



Validación y recarga de Nginx

Editar, validar, recargar y comprobar

1. Validación

Comprueba la sintaxis y atiende al mensaje de error antes de aplicar los cambios.

```
docker compose exec web nginx -t
```

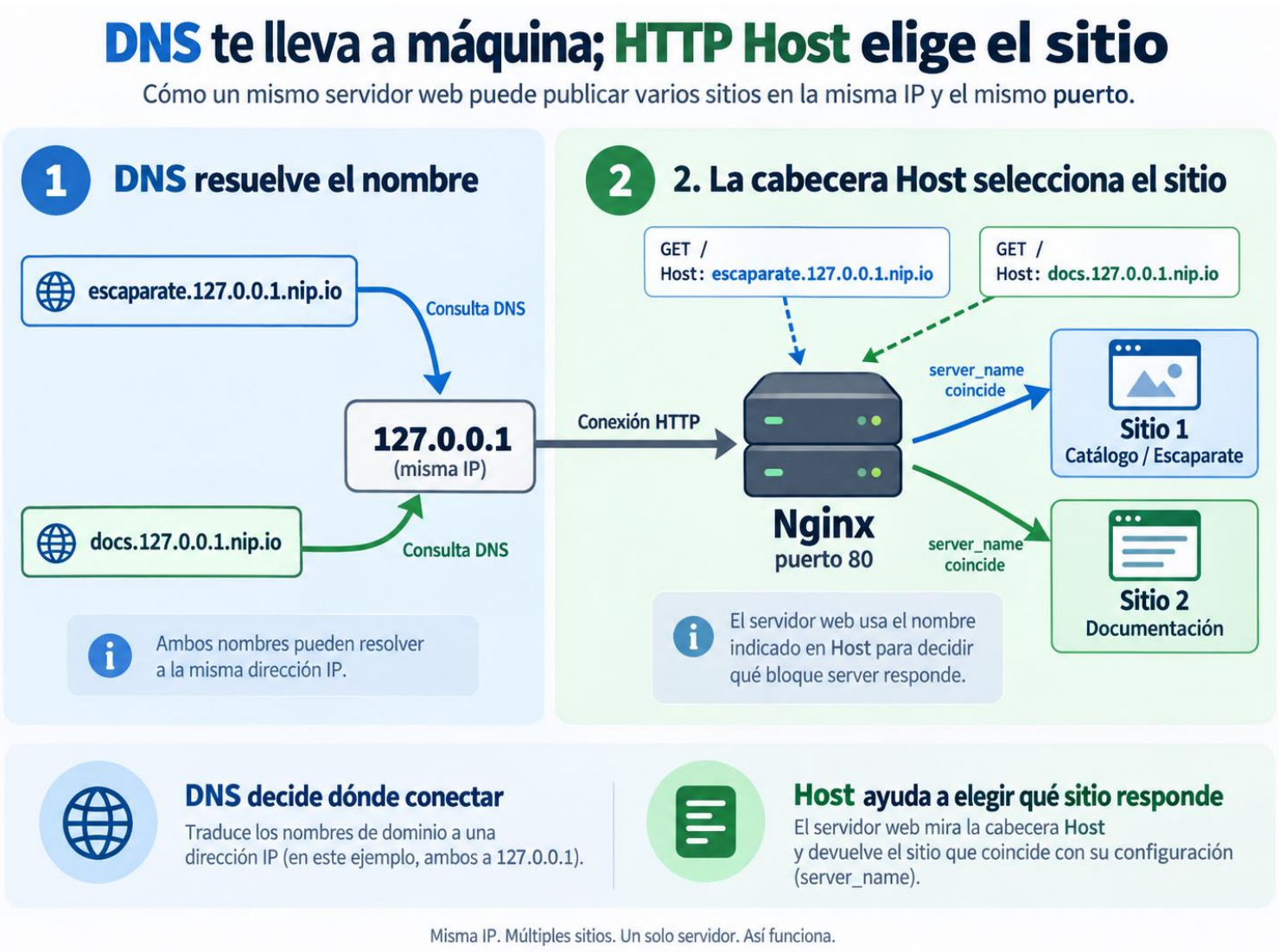
2. Recarga

Si la validación es correcta, recarga y comprueba la respuesta del sitio.

```
docker compose exec web nginx -s reload
```

La recarga aplica una configuración válida sin recrear el contenedor.

DNS y Host seleccionan el destino



Resolución DNS en el laboratorio

Registros y caché DNS

A: relaciona un nombre con una IPv4.

CNAME: define un alias de otro nombre.

TTL: tiempo de conservación de la respuesta en caché.

nip.io en esta práctica

docs.127.0.0.1.nip.io

Resuelve a 127.0.0.1 sin registrar un dominio propio.

```
dig web.127.0.0.1.nip.io
dig +short web.127.0.0.1.nip.io
```

dig consulta DNS directamente. No utiliza /etc/hosts para resolver el nombre solicitado.

Hosts virtuales en Nginx

```
server {
    listen 80;
    server_name docs.127.0.0.1.nip.io;
    root /srv/www/docs;
    index index.html;

    location /informes/ {
        autoindex on;
    }
}
```

listen: puerto de escucha

server_name: nombre del sitio

root: raíz de documentos

index: fichero de entrada

location: reglas por ruta

Cada bloque server combina el nombre del sitio con su raíz de documentos.

Nombres desconocidos y sitio por defecto

Un nombre que no coincide necesita una respuesta prevista

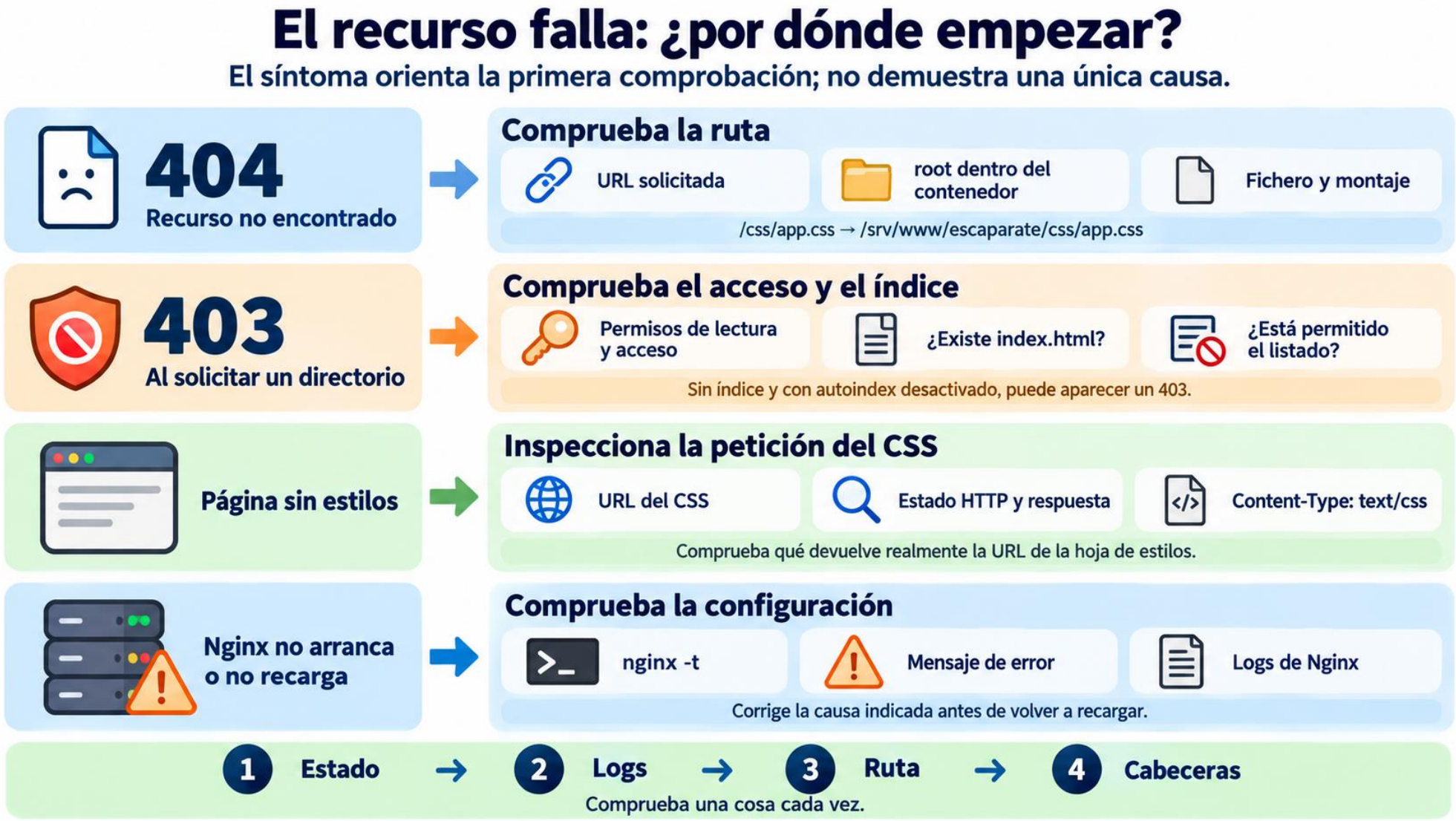
```
server {
    listen 80 default_server;
    server_name _;
    return 404;
}
```

Si ningún server_name coincide, este bloque devuelve 404.

Así evitamos mostrar accidentalmente otro sitio.

default_server fija el bloque de reserva para ese puerto de escucha.

Diagnóstico del contenido estático



Permisos y tipos MIME

Acceso al fichero

Nginx necesita leer el fichero y atravesar sus directorios.

root debe coincidir con la ruta dentro del contenedor.

En directorios, revisa index y autoindex.

Interpretación en el navegador

HTML: text/html

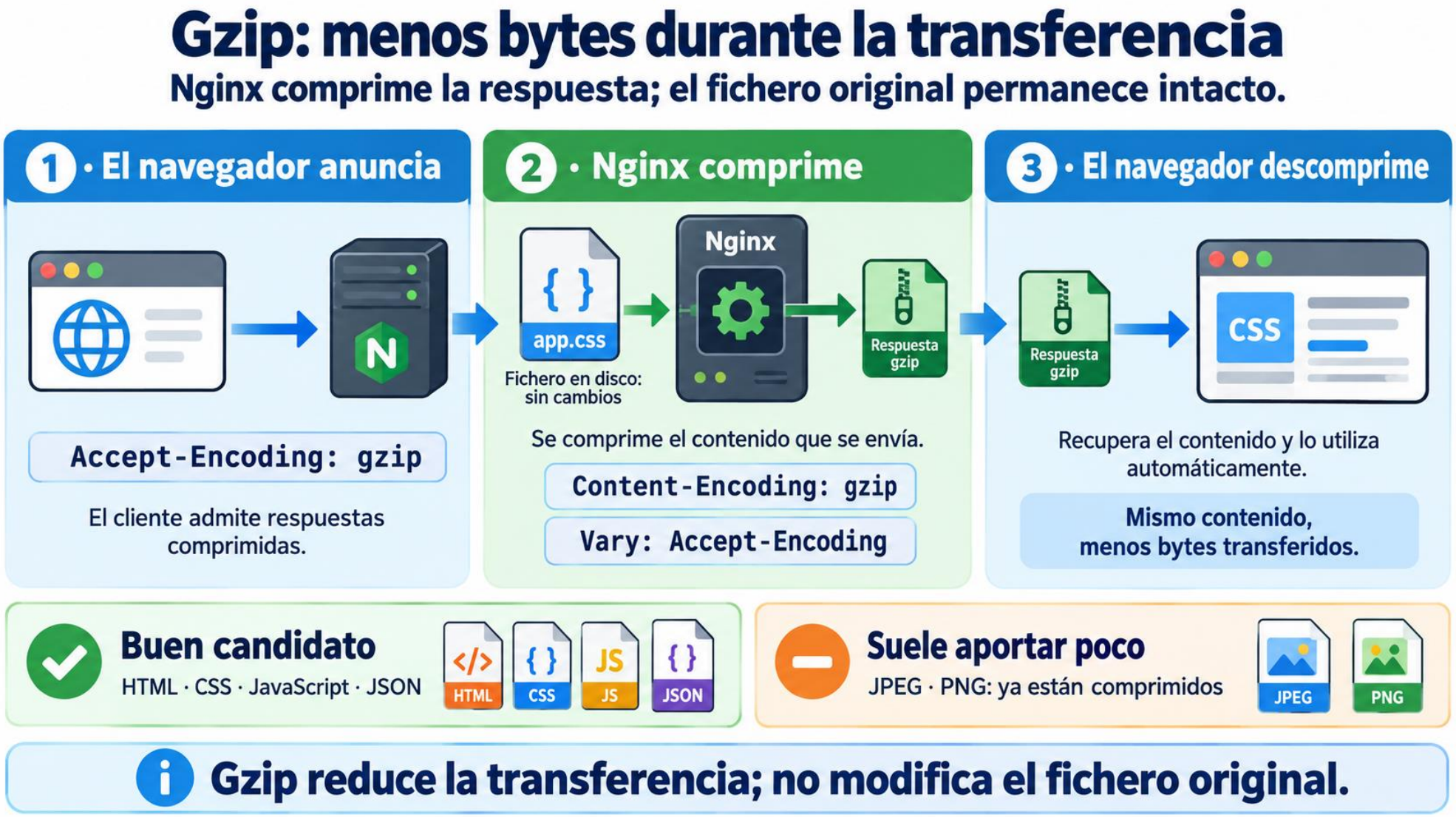
CSS: text/css

JavaScript: application/javascript

PNG: image/png

Un fichero puede existir y descargarse, pero no funcionar con un Content-Type inesperado.

Compresión gzip



Configuración y comprobación de gzip

Configuración de Nginx

```
gzip on;
gzip_vary on;
gzip_types text/css
  application/javascript
  application/json;
```

text/html ya está contemplado por el módulo gzip.

Comprobación de cabeceras

```
curl -I -H "Accept-Encoding: gzip" \

http://escaparate.127.0.0.1.nip.io/css/app.css
```

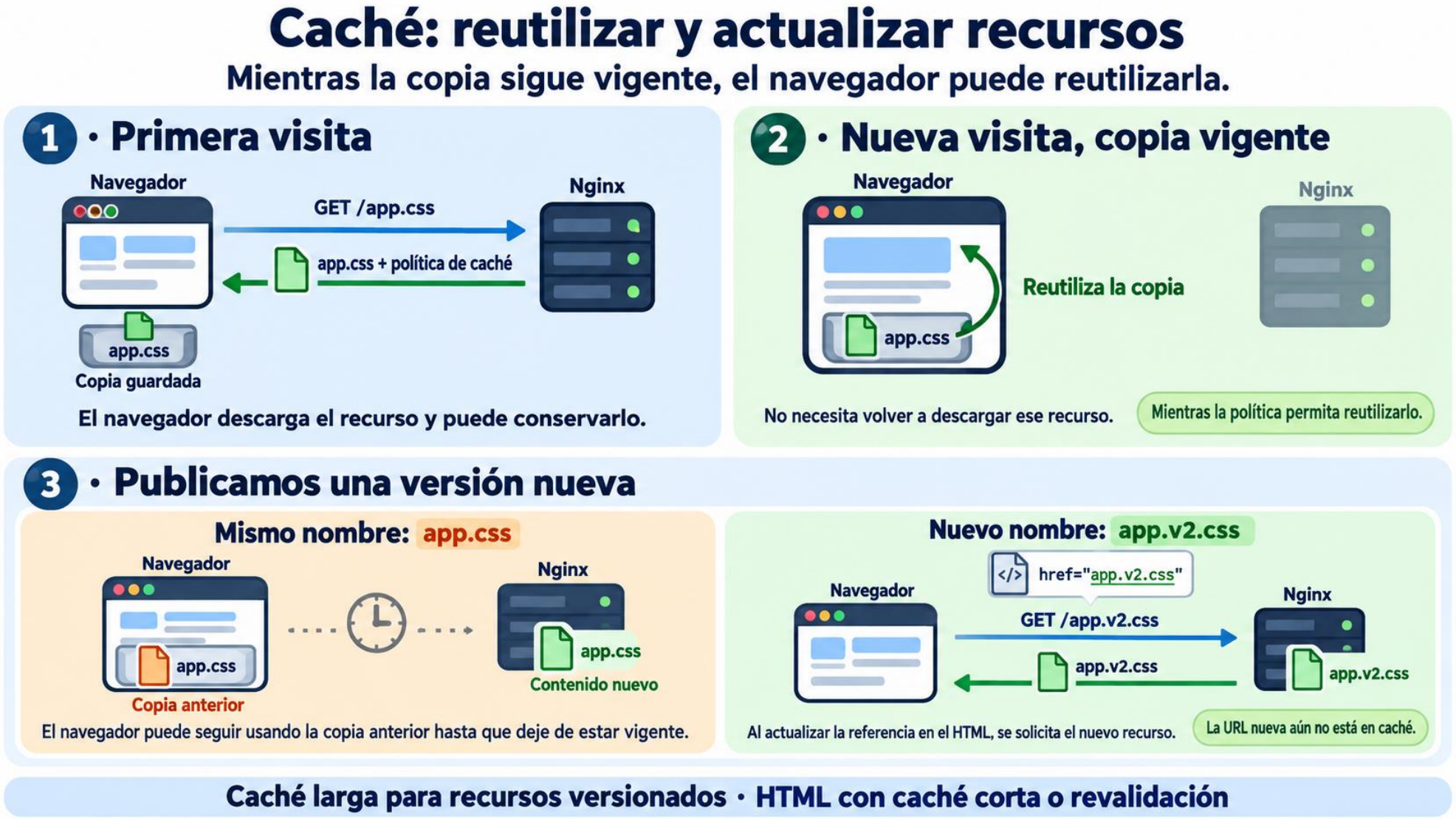
```
Content-Encoding: gzip

Vary: Accept-Encoding
```

Los apuntes incluyen los comandos para comparar el tamaño de descarga.

Para comparar bytes transferidos se necesita una petición completa, no solo cabeceras.

Caché y versionado de recursos



Políticas de caché según el recurso

Políticas habituales

HTML: caché corta o revalidación.

CSS, JS e imágenes versionadas: caché más larga.

API: decidir según el significado y la volatilidad de la respuesta.

Ejemplo para recursos estáticos

```
location ~*
  \.(css|js|png|jpg|jpeg|svg|webp)$ {
    expires 7d;
}
```

Con caché larga, versiona el nombre y actualiza la referencia del HTML.

Una API puede usar caché. La política depende del dato y de cuánto cambia.

Actividad: dos sitios en Escaparate

Aplicación de los conceptos al despliegue

Despliegue

Añadir web y publicar solo su puerto 80.

Montar configuración y contenido.

Configurar catálogo, documentación y servidor por defecto.

Comprobaciones

Validar y recargar Nginx.

Comprobar DNS, Host y rutas.

Verificar gzip y justificar la política de caché.

La siguiente sesión desarrollará el proxy inverso que conecta /api/ con la aplicación.

Cierre de la sesión

Lo mínimo que debe quedar claro

Nginx es la única entrada HTTP pública del despliegue.

root y los montajes determinan dónde están los ficheros.

DNS obtiene la dirección y Host ayuda a seleccionar el sitio.

El diagnóstico se apoya en estado, logs, rutas y cabeceras.

Gzip reduce la transferencia sin modificar el fichero original.

La caché se adapta a cada recurso y requiere planificar las actualizaciones.

Nginx entrega los estáticos y Spring Boot continúa ejecutando la lógica de Escaparate.